



Tecnicatura Universitaria en Programación

Trabajo Práctico Integrador

Programación II

Alumnos - Integrantes:

Aldana, Lara Agustina - Comisión 6
Miglioranza, Manuel Agustín - Comisión 8
Pavic, Lucelia Ines - Comisión 2
Rodriguez, Florencia Mabel - Comisión 1

Coordinador

Martinez, Carlos

Profesores

Cortez, Alberto
Enferrel, Ariel
Rigoni, Cinthia

Tutores

Robledo, Juan
Vega, Marcos
Jerónimo Cortez

Link repositorio: <https://github.com/FlorR566/TPI-Programacion2>

Link video: https://youtu.be/evJ12b_Eqz4

20 de Junio de 2026

ÍNDICE GENERAL

1. INTRODUCCIÓN Y OBJETIVOS	Página 2
2. MARCO TEÓRICO	Página 2
3. DISEÑO DE ARQUITECTURA Y FLUJO DE DATOS EN MEMORIA	Página 3
4. DESARROLLO E IMPLEMENTACIÓN DE ÉPICAS	Página 4
5. DIFICULTADES Y SOLUCIONES	Página 5
7. CONCLUSIONES	Página 5
8. BIBLIOGRAFÍA	Página 6

1. INTRODUCCIÓN Y OBJETIVOS

El presente informe documenta el diseño e implementación del sistema **Food Store**, una aplicación de consola en Java desarrollada como Trabajo Práctico Integrador para la materia Programación 2. El sistema resuelve de manera integral el backlog de historias de usuario orientado a la gestión de productos, categorías, usuarios y el procesamiento dinámico de pedidos dentro de un negocio gastronómico.

Los objetivos principales de este desarrollo radican en afianzar los conceptos fundamentales del paradigma orientado a objetos, asegurar la robustez de las interfaces mediante la sanitización estática de entradas de datos, y aplicar mecanismos eficientes de manejo de colecciones y excepciones personalizadas en memoria antes de la transición hacia la capa de persistencia relacional.

2. MARCO TEÓRICO

El desarrollo del sistema se fundamenta en los pilares de la Programación Orientada a Objetos (POO), la organización arquitectónica de responsabilidades y el diseño de software tolerante a fallos:

- **Encapsulamiento:** Mecanismo que oculta los detalles de implementación de un objeto, protegiendo su estado interno de modificaciones externas arbitrarias. Se implementa declarando los atributos de las entidades como privados (`private`), restringiendo su acceso y mutación exclusivamente a métodos públicos (`getters` y `setters`).
- **Abstracción:** Proceso mental que aísla las características esenciales de un objeto del mundo real para modelarlo en el sistema. Las clases `Usuario`, `Pedido` y `Producto` son abstracciones funcionales orientadas a un contexto gastronómico. Nos basamos en el diagrama UML provisto para la implementación concreta de cada clase.
- **Asociación Unidireccional Efectiva:** Relación estructural que describe una conexión entre objetos donde el origen posee una referencia explícita del destino (ej: `Producto` contiene un atributo `Categoria`), permitiendo la navegación en un solo sentido. Esto desacopla las clases y evita referencias circulares.
- **Interfaces:** Colección de métodos abstractos que definen un contrato de comportamiento obligatorio. Al implementar la interfaz `Calculable` en `Pedido`, se garantiza polimorfismo y una firma estandarizada para el cálculo de totales comerciales mediante el método `calcularTotal`.
- **Gestión Dinámica de Memoria (Collections API):** Estructuras como `ArrayList` manejan un arreglo dinámico modificable en tiempo de ejecución. Permiten inserciones y búsquedas eficientes sin las restricciones de tamaño fijo de los arrays tradicionales.
- **Manejo de Excepciones de Negocio:** Mecanismo estructurado para desviar el flujo normal del programa ante eventos inesperados. Las excepciones propias extienden la clase `Exception` y representan errores semánticos (ej: falta de existencias), permitiendo detener procesos inconsistentes antes de alterar los datos.

3. DISEÑO DE ARQUITECTURA Y FLUJO DE DATOS EN MEMORIA

Para el sistema *Food Store*, se optó por un diseño de responsabilidades distribuidas centrado en componentes operacionales y almacenamiento en memoria volátil. Dado que la aplicación no cuenta en esta fase con persistencia externa, la arquitectura se diseñó para que la interfaz de usuario interactúe con servicios que manipulan los datos en memoria:

src/foodstore/

|— Main.java

| → Punto de entrada de la aplicación

|— entities/

| → Capa de dominio (modelo de datos)

| |— Base.java

| |— Categoria.java

| |— DetallePedido.java

| |— Pedido.java

| |— Producto.java

| |— Usuario.java

|— enums/

| → Enumeraciones del sistema

| |— Estado.java

| |— FormaPago.java

| |— Rol.java

| |— TipoValidacion.java

|— interfaces/

| → Definición de comportamientos

| |— Calculable.java

|— utils/

→ Utilidades generales

|— Validador.java

4. DESARROLLO E IMPLEMENTACIÓN DE ÉPICAS

Cada épica del backlog obligatorio tiene su correspondencia con las clases implementadas.

Gestión de Categorías y Productos (Épica 1 y 2)

- Paquete `entities/`: Modela las clases `Producto` y `Categoria`. `Producto` posee un atributo del tipo `Categoria`, materializando una relación 1→N unidireccional efectiva.
- Paquete `utils/`: Valida precios y existencias mediante `Validador.esDigitoNoNegativoValido()` y `Validador.esNumeroEnteroNoNegativoValido()`, aceptando valores límite y bloqueando textos o números negativos.

Gestión de Usuarios (Épica 3)

- Paquete `entities/` y `enums/`: La clase `Usuario` define usuarios utilizando un tipo enumerado (`Enum`) en lugar de cadenas libres, asegurando perfiles fijos dentro del sistema.
- Paquete `utils/`: Filtra los nombres con `Validador.validarCadena()`, bloqueando aquellas cadenas vacías.

Gestión de Pedidos y Detalles (Épica 4)

- Paquete `entities/`: La entidad `Pedido` mantiene una relación composición con `DetallePedido`, manejando todo su ciclo de vida y agregándolo en memoria mediante el método `addDetallePedido`. Implementa obligatoriamente la interfaz `Calculable`, sobrescribiendo `calcularTotal()` para computar dinámicamente la suma de los subtotales (cantidad x precio).
- Paquete `utils/`: Valida que las cantidades ingresadas por el operador mediante `esNumeroEnteroValido()` sean estrictamente mayores a cero.
- Para eliminar un pedido sin perder la el registro histórico, se altera el atributo booleano a `eliminado = true`. Los métodos del servicio filtran las consultas para ocultar estas órdenes de los listados comunes, manteniéndolas en memoria pero con ese atributo modificado para filtrarlas más fácilmente.

DIFICULTADES ENCONTRADAS Y SOLUCIONES IMPLEMENTADAS

Conflictos de Sincronización al Trabajar sobre la Rama `main`:

- *Dificultad:* Al inicio del proyecto, las subidas directas a la rama principal provocaban constantes conflictos de código (*merge conflicts*), sobreescritura de archivos y pérdida de estabilidad en la aplicación.
- *Solución:* Se implementó un flujo de trabajo basado en ramas independientes de características (*feature branches*) y se trabajó en modalidad “pair programming”. Cada funcionalidad se programó por separado, se testeó localmente, se solicitó un review concreto en los pull request y una vez aprobado, se integraban a `main` de manera controlada y fácilmente reversible en caso de errores o conflictos.

CONCLUSIONES

El desarrollo del sistema *Food Store* permitió consolidar de forma práctica los fundamentos del paradigma orientado a objetos bajo el lenguaje Java. El diseño de una solución distribuida en paquetes funcionales independientes demostró ser un enfoque eficiente para aislar la lógica de negocio de las interfaces de entrada y salida. Asimismo, la correcta implementación de mecanismos de sanitización estática a través de la clase `Validador`, combinada con la robustez de las excepciones personalizadas y el dinamismo de las colecciones en memoria, garantizó el despliegue de una aplicación tolerante a fallos, escalable y alineada con los criterios de calidad académica exigidos.

A nivel técnico y diseño de clases, no se encontraron dificultades que hayan impedido el normal desarrollo del trabajo. El material de clase fue un recurso muy útil al momento de despejar dudas así como la investigación y resolución de dudas mediante búsquedas en internet y páginas de especialización (StackOverflow principalmente).

En cuanto la dinámica de trabajo, se aprovechó la modalidad de “pair programming” en las reuniones virtuales ya que se ajustaba mejor a la disponibilidad y aptitudes de cada uno de los integrantes del equipo, facilitando el intercambio de ideas y el conocimiento diverso sobre los distintos temas vistos en la cursada. Asimismo se trabajó con distintas ramas para evitar los conflictos en los cambios realizados para cada una de las épicas indicadas en la consigna principal.

BIBLIOGRAFÍA

Texto de cátedra y material complementario, Unidad 1 - Introducción a Java, Universidad Tecnológica Nacional.

Texto de cátedra y material complementario, Unidad 2 - Programación estructurada, Universidad Tecnológica Nacional.

Texto de cátedra y material complementario, Unidad 3 - Introducción a POO, Universidad Tecnológica Nacional.

Texto de cátedra y material complementario, Unidad 4 - POO, Universidad Tecnológica Nacional.

Texto de cátedra y material complementario, Unidad 5 - UML Básico, Universidad Tecnológica Nacional.

Texto de cátedra y material complementario, Unidad 6 - Colecciones, Universidad Tecnológica Nacional.

Texto de cátedra y material complementario, Unidad 7 - Herencia y Polimorfismo, Universidad Tecnológica Nacional.

Texto de cátedra y material complementario, Unidad 8 - Interfaces y Excepciones, Universidad Tecnológica Nacional.

Texto de cátedra y material complementario, Unidad 9 - Genéricos, Universidad Tecnológica Nacional.

Links:

<https://stackoverflow.com/questions/7056959/convert-string-to-equivalent-enum-value>

<https://stackoverflow.com/questions/30414859/how-to-cast-subclass-object-to-superclass-object>

<https://stackoverflow.com/questions/4893558/calling-setters-from-a-constructor>

<https://www.geeksforgeeks.org/java/class-getSimpleName-method-in-java-with-examples/>

<https://www.baeldung.com/java-class-name>

<https://stackoverflow.com/questions/6765024/generic-type-as-parameter-in-java-method>